Programmazione 2

a.a. 2025-2026

Abstract Data Types (ADT)

Alcuni ADT fondamentali

- ★ **lista** (ampiamente studiata come lista linkata)

Da fare:

- ★ **pila** (in inglese: stack)
- ★ **coda** (in inglese: queue)
- ★ **insieme** (in inglese: set)

Pila (stack)

Intuizione: stack (pila)

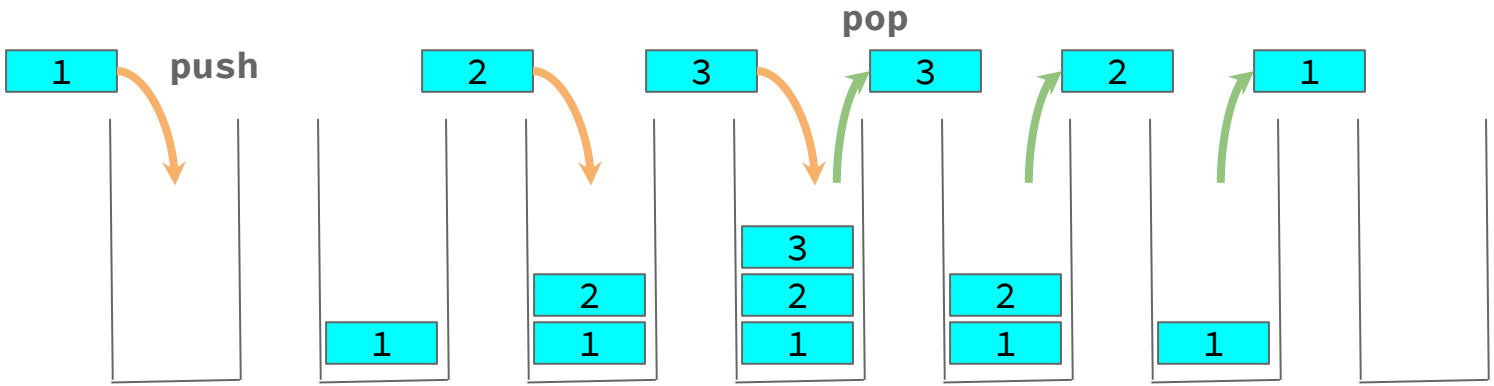
Una pila è una **sovrapposizione di elementi dello stesso tipo**

Per esempio una pila di piatti è costituita da tanti piatti, uno sopra all’altro

Una pila è gestita seguendo la politica **LIFO** (**LAST-IN-FIRST-OUT**)

Intuizione: stack (pila)

LAST-IN-FIRST-OUT (LIFO): si prende/estrae (**pop**) per primo il piatto in cima, che è anche quello inserito (**push**) in pila per ultimo



Esempio: stack di esecuzione

Uno stack di esecuzione è una pila i cui elementi sono record di attivazione

- ★ Ogni **chiamata di funzione** fa il **push** di un nuovo record di attivazione
- ★ Ogni **exit** fa un **pop**
- ★ La funzione invocata più di recente è l’ultima di cui è stato fatto il push ed è la prima di cui si fa pop

ADT Stack: operazioni

Push: aggiungi un elemento in cima allo stack

Pop: rimuovi un elemento dalla cima dello stack

Empty: verifica se lo stack è vuoto

Full: verifica se lo stack è pieno [se limitato]

Peek: ottieni il valore in cima allo stack senza rimuoverlo

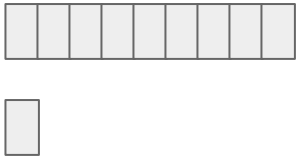
Interfaccia

- ★ void **push**(tipo-dato el, stack *s)
- ★ tipo-dato **pop**(stack *s)
- ★ int **empty**(stack* s)
- ★ int **full**(stack* s)
- ★ tipo-dato **peek**(stack* s)

NB: in C il tipo int è usato anche per i valori booleani, in particolare 0 corrisponde a falso, e un valore diverso da zero corrisponde a vero. In alternativa si può usare _Bool, o ancora (includendo stdbool.h) bool (abbreviazione di _Bool), true (=1), false (=0)

Implementazioni

TRAMITE ARRAY

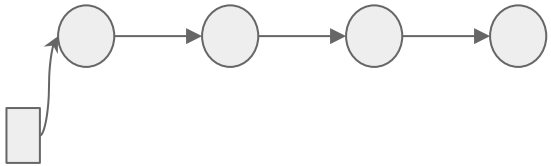


L’array ha una capienza predefinita

L’accesso avviene tramite indice

Serve una variabile (top) che ci dica fin dove l’array è pieno. Ogni push/pop deve aggiornare top

TRAMITE LISTA LINKATA



La lista può crescere senza limiti

push/pop = inserzione/estrazione dalla testa

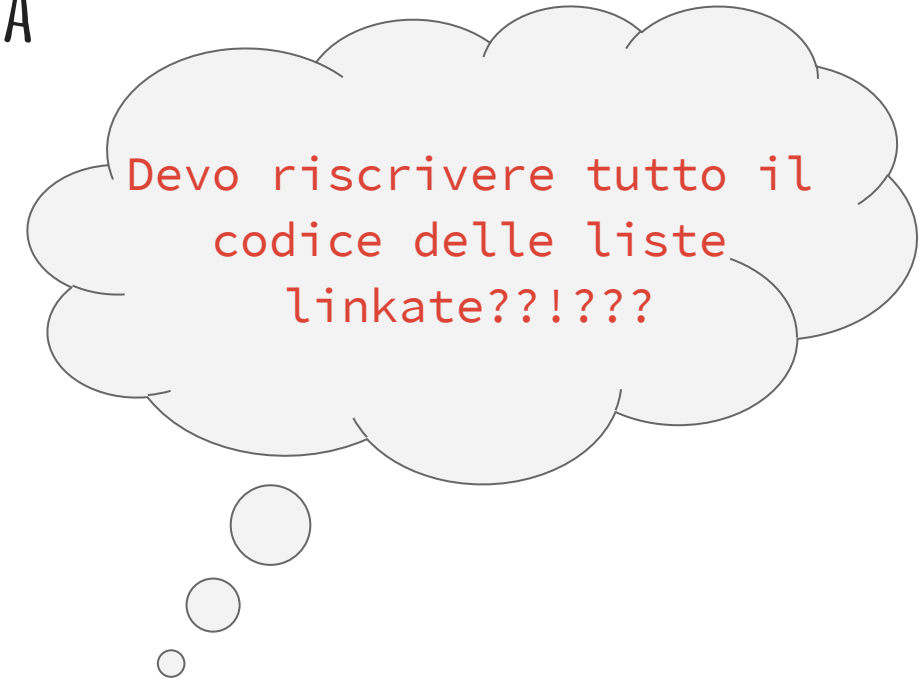
L’elemento in coda è il primo a essere stato inserito (il più "profondo" nello stack)

Implementazione tramite lista

```
typedef struct nodo *lista;
struct nodo {
    tipo-dato  dato;
    lista *next;
}
typedef lista stack;
stack mio_stack;
```

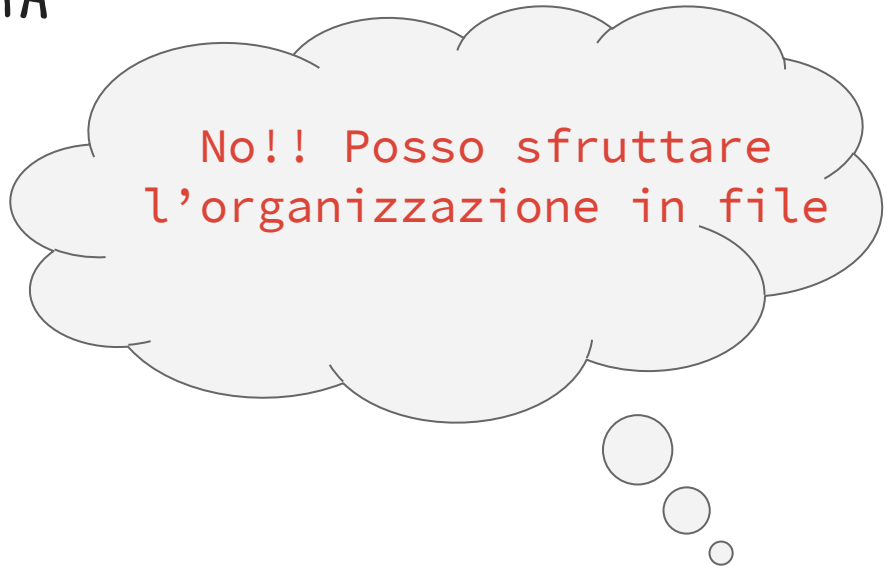
Implementazione tramite lista

```
typedef struct nodo *lista;
struct nodo {
    tipo-dato  dato;
    lista *next;
}
typedef lista stack;
stack mio_stack;
```



Implementazione tramite lista

```
typedef struct nodo *lista;
struct nodo {
    tipo-dato  dato;
    lista *next;
}
typedef lista stack;
stack mio_stack;
```



Implementazione tramite lista

```c
#include <stdio.h>

#include <stdlib.h>


#include "lista.h"


typedef lista stack;
```

lista.h

```c
typedef struct nodo* lista;
struct nodo {
    int dato;
    lista next;
};

lista crea_nodo(int);
void stampa(lista);
void ins_testa(int, lista*);
lista del_testa(lista*);
```

Implementazione tramite lista

```
#include <stdio.h>

#include <stdlib.h>


#include "lista.h"


typedef lista stack;
```

uno stack è realizzato come una lista speciale, che sarà usata tramite le funzioni dichiarate nel seguito

Implementazione tramite lista

```
int empty(stack* s) {
    return !(*s);  // se stack vuoto, (*s) == NULL
}
int full(stack* s) {
        return 0;  // in questa implementazione
}                  // lo stack non ha un numero massimo
                   // di elementi
```

Implementazione tramite lista

```
void push(int el, stack *s) {
    if (s) // se il parametro è definito
        ins_testa(el, s);
        // il nodo è creato da ins_testa
}
```

NB: in questa implementazione se lo stack esiste la push è sempre possibile

Implementazione tramite lista

```
int pop(stack *s) {
    lista top; int result;
    if (s && !empty(s)) {  // controlli di rito
       top = del_testa(s);
       result = top->dato;
        free(top);
    }
    else result = NOVALUE;
    return result;
}
```

NOVALUE

NOVALUE è una costante definita ad hoc a seconda del range dei dati significativi. L'ambiente chiamante deve verificare se il dato è significativo o meno:

```
if (result != NOVALUE) … gestione dato …
```

Se non si può determinare un valore adeguato occorre restituire due valori: uno che corrisponde al dato, l'altro che dice se quel dato è significativo oppure no

Implementazione tramite lista

```
int peek(stack* s) {
    int result = NOVALUE;
    if (s && !empty(s)) // controlli di rito
        result = (*s)->dato;
    return result;
}
```

COMPILAZIONE DEL PROGRAMMA

```
>> gcc -o stack stack.c lista.c
```

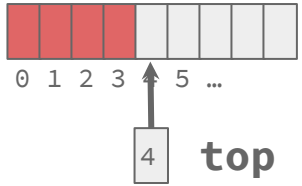
Ricordate?

quando il programma è diviso per parti la linea di compilazione deve comprendere l’elenco di tutti i sorgenti

Implementazione tramite array: ricominciamo!

```
#define NOVALUE -1000

#define MAX 50


typedef struct {

        top;

        int dato[MAX];

} *stack;
```

questa volta occorre prevedere una capienza massima per lo stack



il campo top indica la prossima cella dell'array in cui fare push

si possono pensare implementazioni alternative in cui top indica la cella in cima alla parte occupata dello stack. In questo caso va inizializzata a -1

Implementazione tramite array

Una nuova funzione:

```
void init(stack s) {
    if (s) s->top = 0;
    else
      printf("\n s non esiste \n");
}
```

da usare per impostare il valore di top e indicare che lo stack è vuoto

```
int empty(stack s) {
     return (s->top == 0);
     // se stack vuoto, top == 0
}

int full(stack s) {
    return (s->top == MAX);
}
```

Implementazione tramite array

```
void push(int el, stack s) {
    if (!full(s)) {  // se non è pieno
        s->dato[s->top] = el;  // aggiorna i campi
        s->top = s->top+1;
    }
    else printf("\n stack pieno\n");  // informa utente
}
```

Implementazione tramite array

```
int pop(stack s) {
    int result = NOVALUE;  // inizializza risultato
    if (!empty(s)) {  // se non è vuoto
        result = s->dato[s->top -1]; // aggiorna risultato
        s->top = s->top - 1;         // aggiorna top
    }
    else printf("\n stack vuoto");   // informa utente
    return result; // restituisci risultato
}
```

Implementazione tramite array

Simile alla precedente, un po’ più semplice

```
int peek(stack s) {
    int result = NOVALUE;
    if (!empty(s))
            result = s.dato[s.top -1];
    else printf("\n stack vuoto");
    return result;
}
```

Principali applicazioni degli stack

Struttura dati molto semplice ma molto “potente”:

- ★ **Esecuzione dei processi** (già studiato)
- ★ **Inversione di una stringa**: sequenza di push delle singole lettere su uno stack e (terminata la stringa) sequenza di pop. L’ordine inverso è ottenuto grazie alla natura LIFO dello stack
- ★ **Compilatori**: calcolo del valore di espressioni (es. 2 + 4 / 5 * (7 - 9)) convertendole in forma prefissa o postfissa
- ★ **Browser**: il pulsante back di un browser salva tutti gli URL visitati in uno stack. Ogni visita di una nuova pagina è aggiunta con push, ogni click su back è un pop che porta alla pagina visitata in precedenza
- ★ **Sistemi operativi**: implementazione delle politiche di sostituzione delle pagine “least recently used”

Coda (queue)

Intuizione: coda in biglietteria

Come una lista una **coda o è vuota oppure è costituita da un elemento seguito da un altro elemento ecc.**

Per esempio in una coda in biglietteria una persona è seguita da un’altra persona e così via

Quindi tutti gli elementi della coda sono persone ma tali persone sono diverse fra di loro

PARTICOLARITÀ: gli elementi della coda sono gestiti **FIRST-IN-FIRST-OUT (FIFO)**

ADT CODA

Una coda è un particolare tipo di sequenza in cui gli elementi sono inseriti ad un capo e sono rimossi dal capo opposto

La gestione è di tipo **FIFO** (First In First Out): il primo che entra è il primo che esce

Operazioni:

- ★ **enqueue:** inserisce un elemento in fondo (tail) alla coda
- ★ **dequeue:** rimuove un elemento dalla testa (head) della coda

Interfaccia

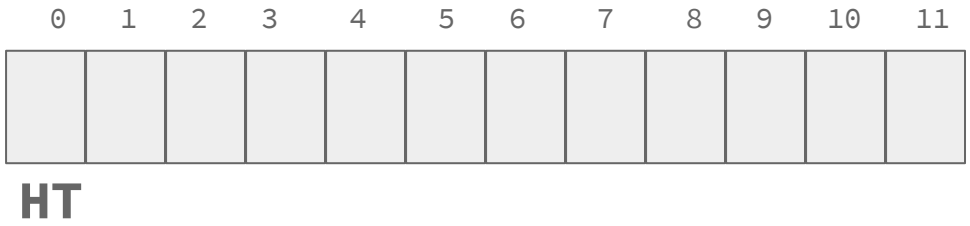
- ★ void **enqueue**(tipo-dato el, coda Q)
- ★ tipo-dato **dequeue**(coda Q)
- ★ int **full**(coda Q)
- ★ int **empty**(coda Q)
- ★ int **peek**(coda Q)
- ★ void **init**(coda Q)

Implementazioni

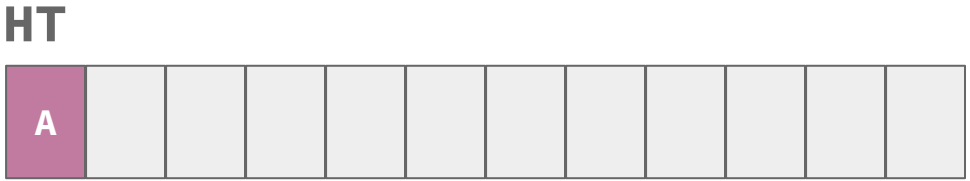
Come sempre con gli ADT non esiste un modo unico di implementarli

Anche in questo caso vedremo sia una soluzione che si appoggia alle liste sia una soluzione che si appoggia agli array, cominciamo con gli array

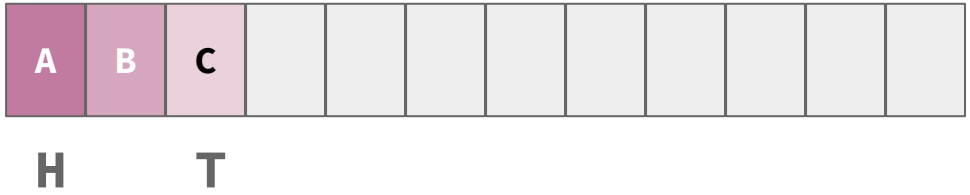
Enqueue



head = 0
tail = 0
num = 0

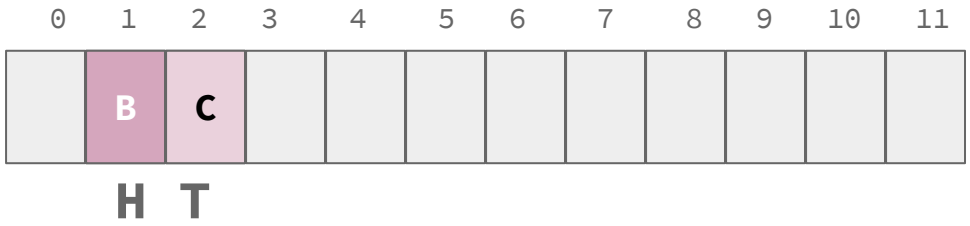


head = 0
tail = 0
num = 1



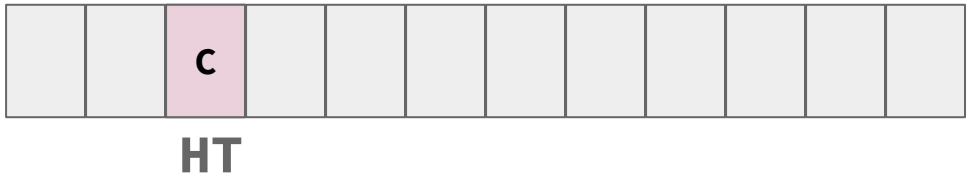
head = 0
tail = 2
num = 3

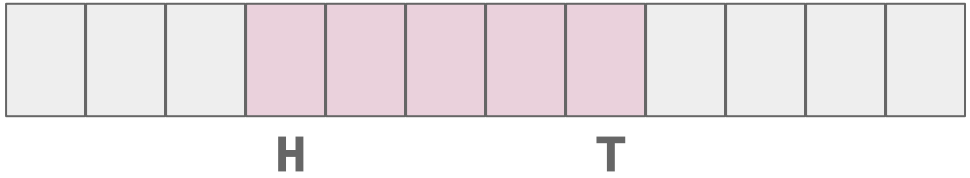
Dequeue



head = 1
tail = 2
num = 2

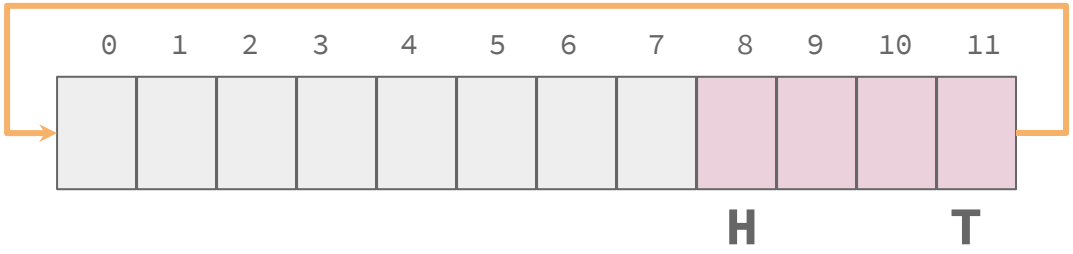
head = 2
tail = 2
num = 1



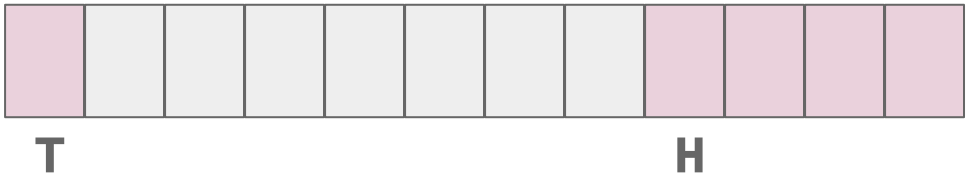


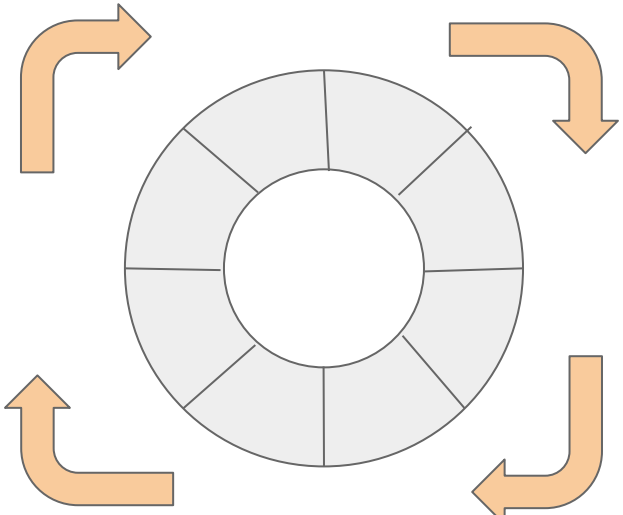
head = h
tail = t
num = (t-h+1)

Enqueue: quando sono in fondo?



head = 8
tail = 11
num = 4





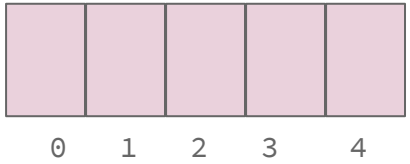
Torno all'inizio se no non riesco a usare parte delle celle e spreco spazio:

coda circolare

Serve uno strumento: modulo

N % k: **N modulo k**, resto della divisione intera di N per k.
Si usa per calcolare il successore in un array circolare

Esempio: supponiamo di avere un array di 5 celle, gli indici vanno da 0 a 4




```
int successore(i, N) {
        return ((i+1)%N);
}
```

```
successore(0, 5)  1
successore(1, 5)  2
…
successore(4, 5)  0
```

Implementazione circolare

```
#define MAX 5
#define NOVALUE -10000

typedef struct {
    char dato[MAX];
    int head; // pos head
    int tail; // pos tail
    int num;
} queue;
```

```
void init(queue* Q) {
    Q->head = 0;
    Q->tail = 0;
    Q->num = 0;
}
```

Implementazione circolare

```
int empty(queue* Q) {
    return (Q->num == 0);
}

int full(queue* Q) {
    return (Q->num == MAX);
}
```

```
int succ(int i) {
   return (i+1)%MAX;
}
```

Enqueue

```
void enqueue(int el, queue* Q) {
   if (!full(Q)) {
       if(!empty(Q)) Q->tail = succ(Q->tail);
       Q->dato[Q->tail] = el;
       (Q->num)++;
   }
   else printf("\n la coda è piena\n");
}
```

Dequeue

```
int dequeue(queue* Q) {
  int result = NOVALUE;
  if (!empty(Q)) {
     result = Q->dato[Q->head];
     (Q->num)--;
     if(!empty(Q)) Q->head = succ(Q->head);
  }
  else printf("\n Q vuota\n");
  return result;
}
```

Empty, Full, Peek (fare per esercizio)

empty e full:

usano il numero di elementi memorizzato nella stessa coda

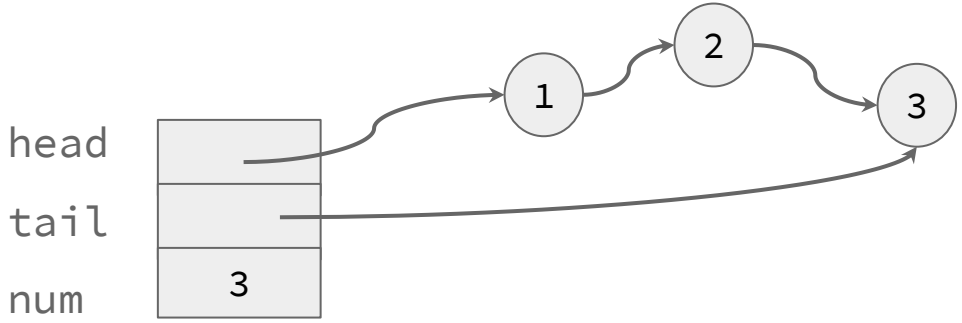
peek:

si realizza in maniera simile a dequeue ma non modifica la coda, si limita a restituire un valore

Implementazione tramite lista

Sfruttiamo le liste già implementate

```
#include “lista.h”
```



Implementazione tramite lista

Sfruttiamo le liste già implementate

```
#include “lista.h”
typedef struct {
        lista head;   // riferimento alla testa (dequeue)
        lista tail;   // riferimento alla coda (enqueue)
        int num;      // eventuali dati extra
} queue;
```

INIT

CHIAMANTE:

```
queue myqueue;
init(&myqueue);
```

FUNZIONE:

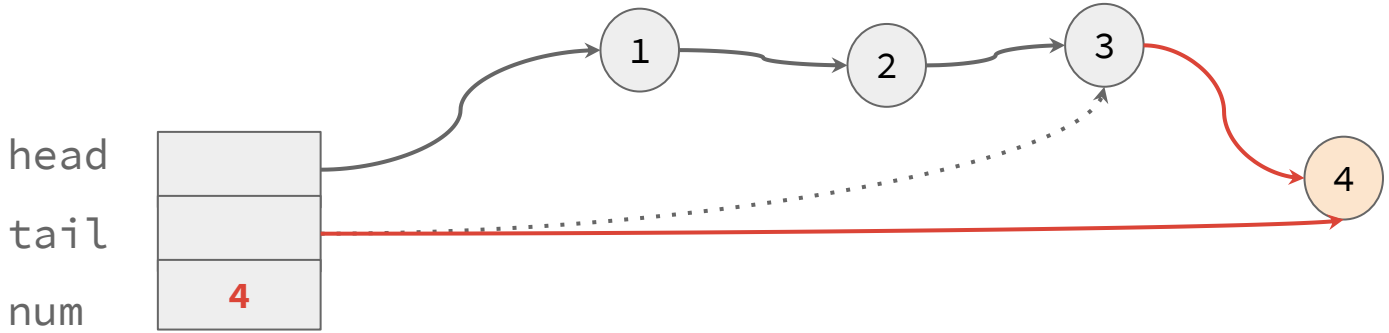
```
void init(queue *Q) {
        Q->head = NULL;
        Q->tail = NULL;
        Q->num = 0;
}
```

head	NULL
tail	NULL
num	0

Enqueue

Aggiunge un elemento in coda tramite tail

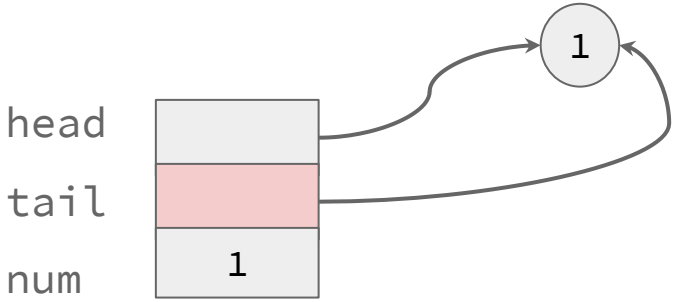
Aggiorna il contatore num



queue myqueue;

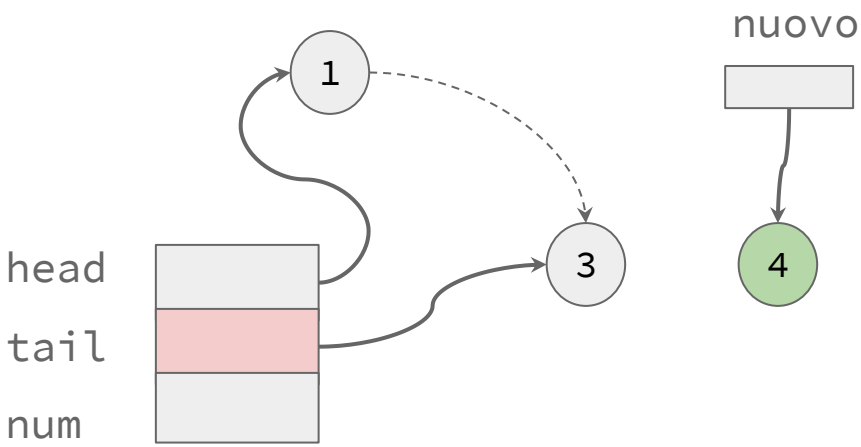
Enqueue: il primo elemento

```
void enqueue(int el, queue *Q) {
  lista nuovo = crea_nodo(el);
  if (Q) // se Q è definita
   if (empty(Q)) { // se la coda è vuota
    Q->head = nuovo;
    Q->tail = nuovo;
    Q->num = 1;
    }
   else { … }
}
```



Enqueue: altri elementi

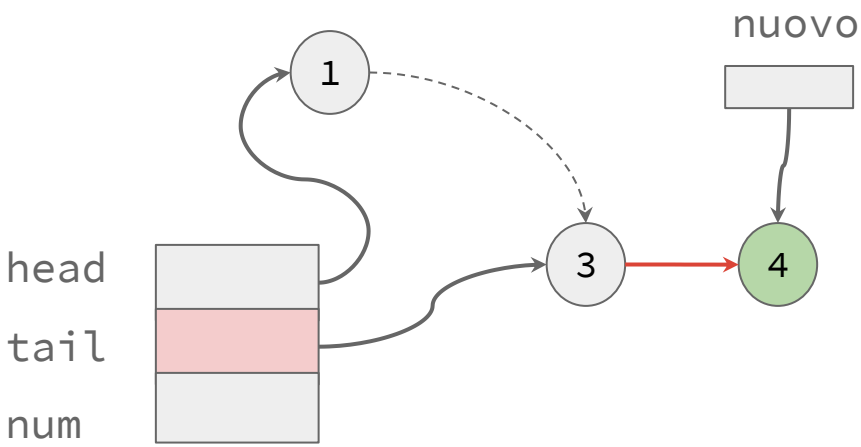
```
void enqueue(int el, queue *Q) {
  lista nuovo = crea_nodo(el);
  if (Q)
   if (empty(Q)) {...}
   else { // caso generale
    Q->tail->next = nuovo;
    Q->tail = nuovo;
    Q->num++;
    }
}
```



Enqueue

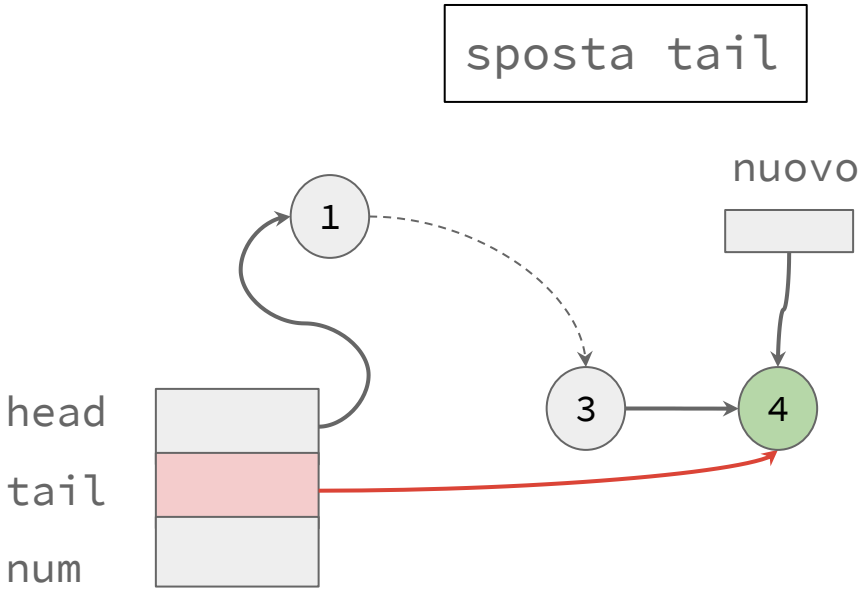
```
void enqueue(int el, queue *Q) {
  lista nuovo = crea_nodo(el);
  if (Q)
   if (empty(Q)) {...}
   else { // caso generale
    Q->tail->next = nuovo;
    Q->tail = nuovo;
    Q->num++;
    }
}
```

aggancia nodo a lista



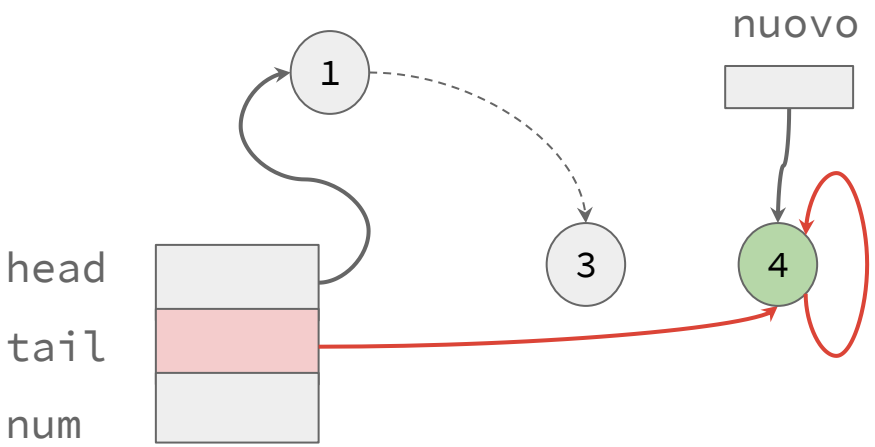
Enqueue

```
void enqueue(int el, queue *Q) {
  lista nuovo = crea_nodo(el);
  if (Q)
   if (empty(Q)) {...}
   else { // caso generale
    Q->tail->next = nuovo;
    Q->tail = nuovo;
    Q->num++;
    }
}
```



NB: L'ORDINE CONTA!!

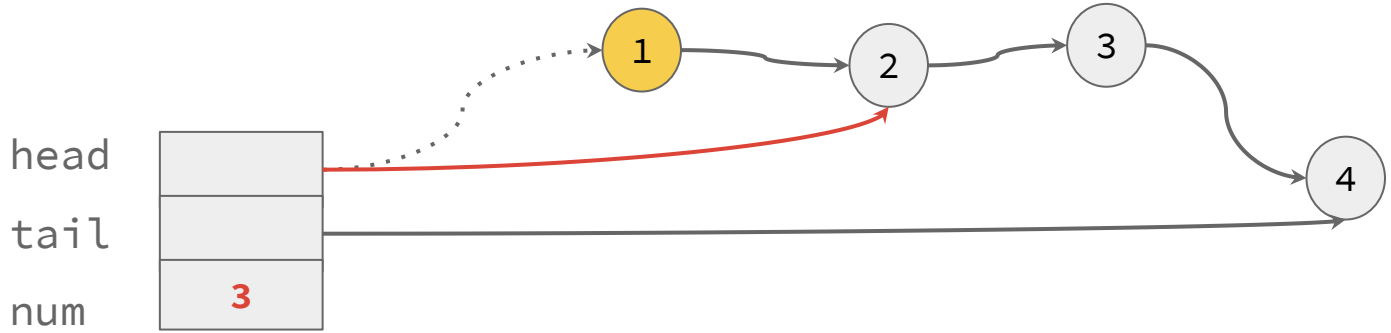
```
void enqueue(int el, queue *Q) {
    …
    Q->tail = nuovo;
    Q->tail->next = nuovo;
    …
}
```



Dequeue

Toglie e restituisce l’elemento in testa tramite head

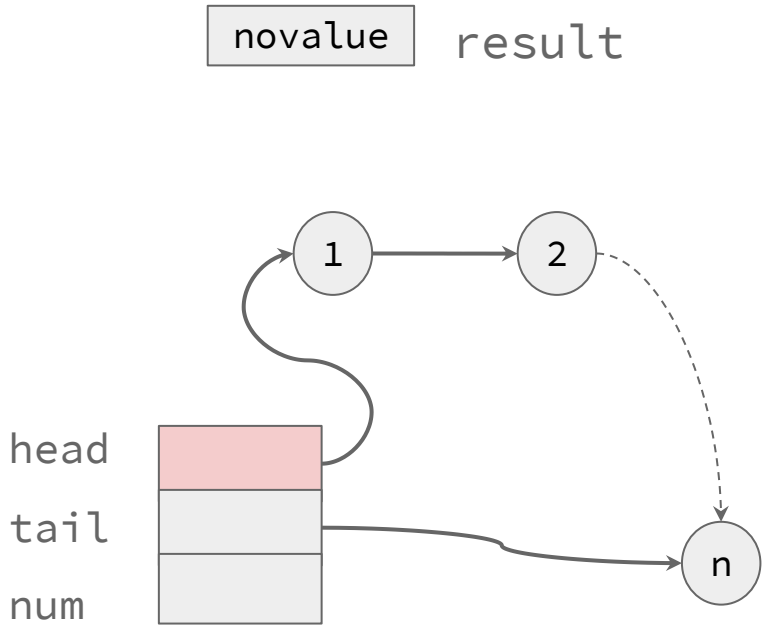
Aggiorna il contatore num



queue Q;

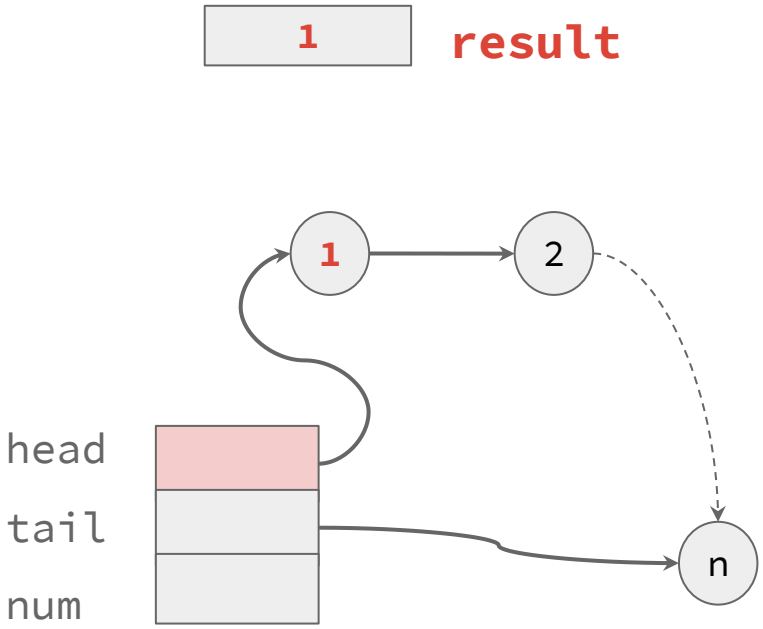
Dequeue

```
int dequeue(queue *Q) {
  int result=NOVALUE;
  if (Q)
    if (!empty(Q)) {
       result = Q->head->dato;
       Q->head=Q->head->next;
     }
  return result;
}
```



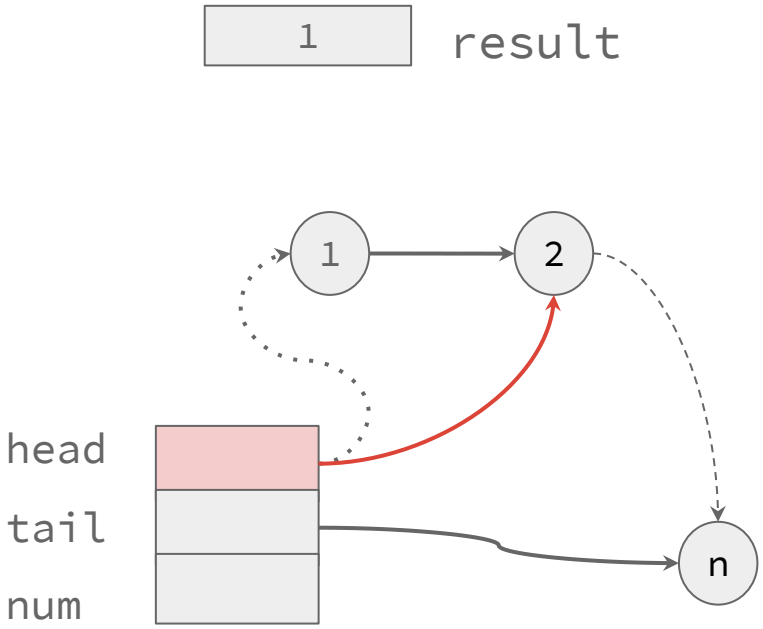
Dequeue

```
int dequeue(queue *Q) {
  int result=NOVALUE;
  if (Q)
    if (!empty(*Q))
      result = Q->head->dato;
      Q->head=Q->head->next;
    }
  return result;
}
```



Dequeue

```
int dequeue(queue *Q) {
  int result=NOVALUE;
  if (Q)
    if (!empty(*Q))
       result = Q->head->dato;
       Q->head=Q->head->next;
     }
  return result;
}
```

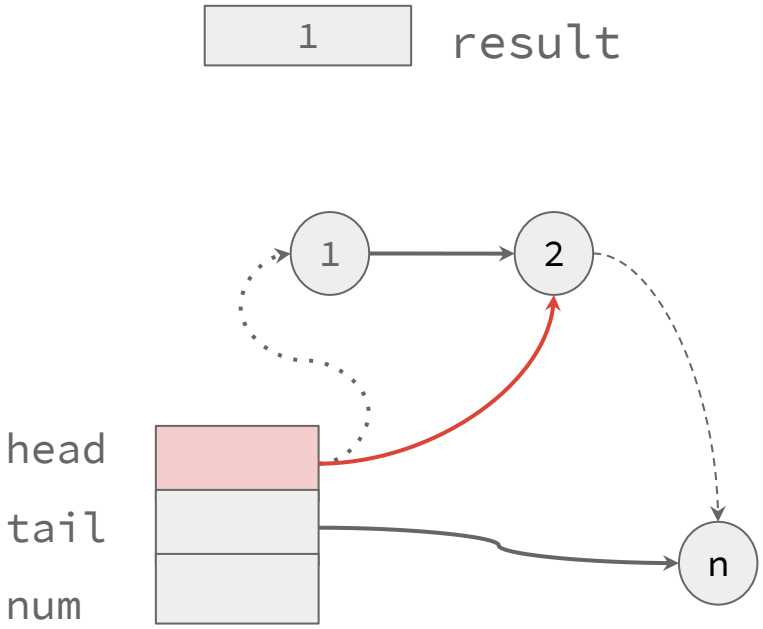


Dequeue

```
int dequeue(queue *Q) {
  int result=NOVALUE;
```

MANCA QUALCOSA??

```
        result = Q->head->dato;
        Q->head=Q->head->next;
      }
  return result;
}
```

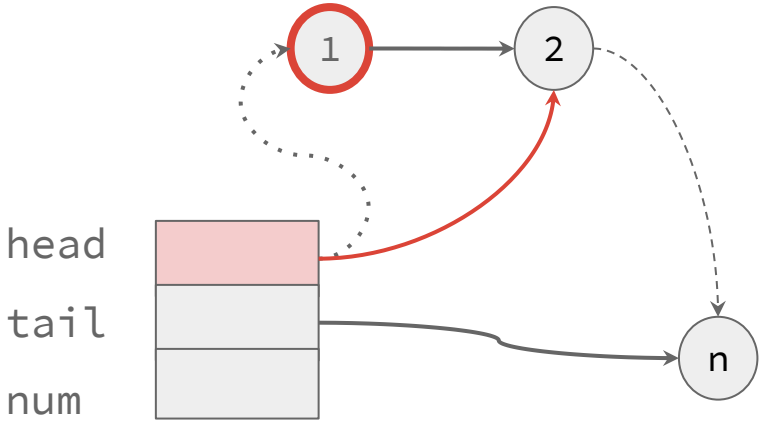


Dequeue

```
int dequeue(queue *Q) {
```

MANCA QUALCOSA??

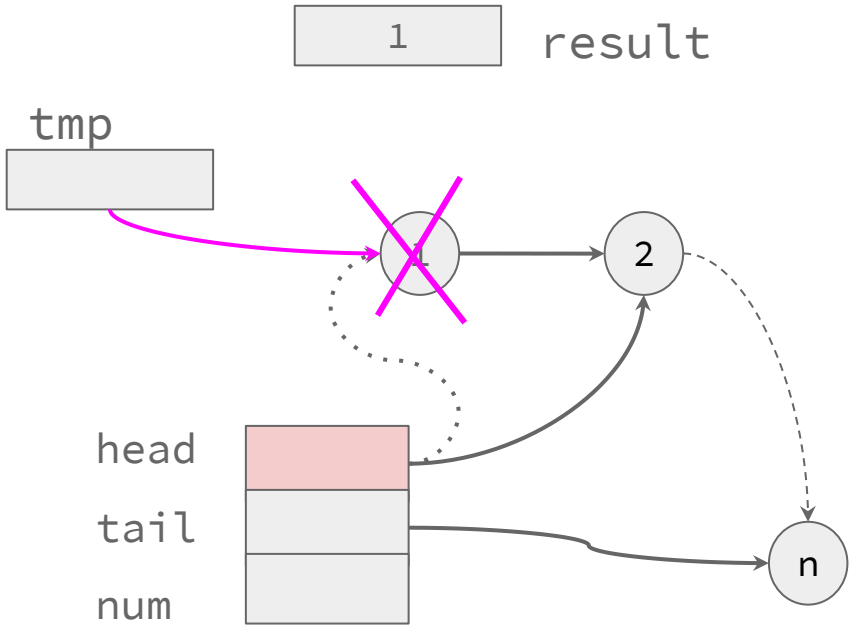
Sì dobbiamo liberare la memoria!!




```
}
```

Dequeue

```
int dequeue(queue *Q) {
  int result=NOVALUE;
  lista tmp;
  if … {
      result = Q->head->dato;
      tmp = Q->head;
      Q->head=Q->head->next;
      free(tmp);
    }
  return result;
}
```



Empty, Full, Peek, Init (fate per esercizio)

Empty: verifica se il campo head vale NULL

Full: in principio non si può riempire, restituisce sempre 0

Peek: simile a dequeue ma si limita a restituire il valore dell'elemento puntato da head, non lo rimuove. La coda viene passata per valore

Alcune applicazioni delle code

- ★ **buffer**: nei sistemi produttore-consumatore i produttori inseriscono dei dati in una coda, i dati sono estratti nell'ordine FIFO dai consumatori, quando sono pronti a elaborarli
- ★ **scheduling della CPU**: il sistema operativo mantiene i processi "ready" (che possono eseguire) in una coda che in alcuni sistemi è gestita FIFO
- ★ **ricerca in ampiezza**: la strategia di ricerca in ampiezza usata nel problem solving si appoggia a una coda FIFO

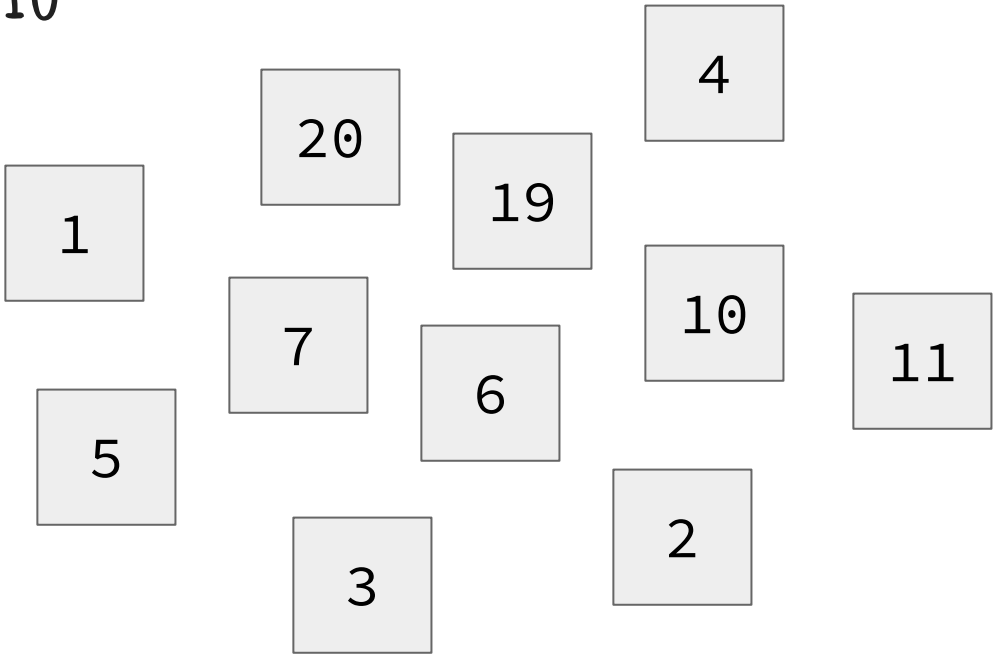
Insieme (set)

Intuizione: insieme (set)

Raccolta di dati priva di ordinamento

Non esistono primo e ultimo

Esempio



INSIEME

Operazioni (esempi)

int **elem**(tipo-dato el, set *S): test di presenza di un elemento

int **vuoto**(set *S): test se vuoto

void **init**(set *S): inizializza a vuoto

int **cardinality**(set *S): num. elem. di S

set ***costruisci**(n, tipo-dato x1,...,xn): crea un set contenente gli n valori x1, x2, …, xn

set ***crea_max**(int N): crea un set di capacità massima N

void **aggiungi**(set *S, tipo-dato el): aggiunge el al set (se non c'è già)

void **togli**(set *S, tipo-dato el): toglie el dal set (se c'è)

Implementazione, esempi

lista linkata

insieme illimitato

inserzione in testa

array

insieme limitato

mi ricordo di quanti elementi ci sono

Set con lista linkata

```
typedef lista set;


set myset;
init(&myset);
// insieme vuoto


void init(set *S) {
   *S = NULL;
}
```

Set con lista linkata

```
typedef lista set;


set myset;

init(&myset);

// insieme vuoto
```

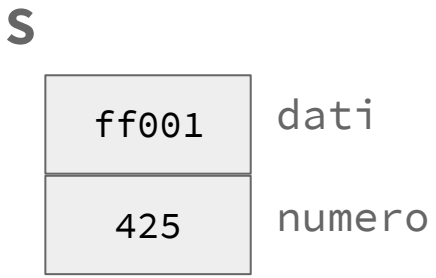
ALTERNATIVA

```
typedef struct {
        lista dati;
        int numero; // metadati
} set;
```

Set con lista linkata e metadati

ALTERNATIVA

```
typedef struct {
        lista dati;
        int numero; // metadati
} set;

set S;
```



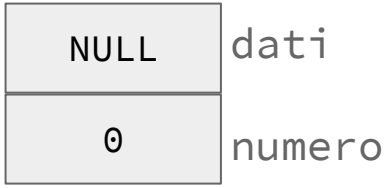
Ricordate: lo spazio di memoria occupato da una variabile appena dichiarata può contenere valori casuali

Set con lista linkata e metadati

ALTERNATIVA

```
typedef struct {
        lista dati;
        int numero; // metadati
} set;


set S;
init(&S);
```

```
void init(set *S) {
        S->dati = NULL;
        S->numero = 0;
}
```



Set con lista linkata e metadati

ALTERNATIVA

```
typedef struct {
        lista dati;
        int numero; // metadati
} set;

set S;
init(&S);
```

```
int vuoto(set* S) {
        …
}

int cardinality(set* S) {
        …
}
```

semplifica, non dobbiamo calcolare ogni volta

AGGIUNGI

```
void aggiungi(set *S, tipo-dato el):
```

E’ sufficiente fare una inserzione in testa?

(**FARE DA SOLI**)

Togli elemento, esempio di soluzione ricorsiva

```
/*
 * Cerca un elemento e se lo trova lo rimuove
 * oltre a togliere l'elemento restituisce un output che serve a capire se l'elemento
 * è stato trovato e rimosso (1), se non c'era (0) oppure se il parametro era mal definito (-1)
 **/

int togli(lista *L, int el) {
    if (L)
            if (*L)
                        if ((*L)->dato == el) {
                                    lista tmp = *L;
                                    *L=(*L)->next;
                                    free(tmp);
                                    return(1);
                        }
                        else {
                                    return(togli(&((*L)->next), el));
                        }
            else return 0;
    else return -1;
}
```

COSTRUISCI

```
set *costruisci(n, tipo-dato x1,...,xn)
```

Allocazione dinamica di un set inizializzato con i valori dati

- ★ Sarebbe bello consentire all'utente di elencare tutti i valori che si desidera aggiungere
- ★ è possibile?

Costruisci

```
set *costruisci(int n, ...) {
    va_list ptr; // puntatore alla lista degli argomenti
    set *nuovo = NULL;
    if (n > 0) {
          nuovo = (set *)malloc(sizeof(set));
          init(nuovo); // ho creato un set vuoto

          // scorri i parametri e inserisci i valori

    };
    return nuovo;
}
```

Costruisci

```
set *costruisci(int n, ...) {
    va_list ptr; // puntatore alla lista degli argomenti
    set *nuovo = NULL;
    if (n > 0) {
            nuovo = (set *)malloc(sizeof(set));
            init(nuovo);
            va_start(ptr, n); // il puntatore fa ora riferimento al
                              // primo elem dei param opzionali
            for (; n>0; n--) {
                    aggiungi(nuovo, va_arg(ptr, int));
            }
            va_end(ptr);
    };
    return nuovo;
}
```